# False Positive Tuning Notes

## Windows Detection Engineering Lab — Wazuh, Sysmon & MITRE ATT&CK

**Project Name:** Windows Detection Engineering Lab — Wazuh, Sysmon & MITRE ATT&CK
**Document Type:** False Positive Tuning Notes
**Environment:** Local VirtualBox Cybersecurity Lab
**Primary Systems:** Windows 10 Lab Endpoint, Ubuntu 22.04 Wazuh Server, Wazuh Dashboard, Sysmon, Windows Event Logs, Windows Defender
**Detection Sources:** Wazuh Custom Rules, Sysmon Telemetry, Windows Security Logs, Windows Defender Events

---

## 1. Document Purpose

The purpose of this document is to identify possible false positives, benign triggers, tuning considerations, and detection-quality improvements for the custom rules created in the **Windows Detection Engineering Lab — Wazuh, Sysmon & MITRE ATT&CK** project.

This document supports the detection engineering lifecycle by explaining not only how detections were created, but also how they should be reviewed, tuned, and improved to reduce unnecessary alert noise while preserving meaningful security visibility.

In a real SOC environment, false positives can reduce analyst trust, increase investigation workload, delay response to real threats, and make high-value alerts harder to identify. For that reason, this project includes a structured false-positive review process for each custom detection.

---

## 2. False Positive Tuning Objective

The main objective of false-positive tuning is to improve detection quality without completely removing useful security coverage.

A successful tuning process should reduce unnecessary alerts while keeping detections sensitive enough to identify suspicious behavior. The goal is not to suppress every benign alert, but to make alerts more accurate, more contextual, and more useful for investigation.

This document aims to achieve the following:

Identify expected benign activity that may trigger each custom detection. Separate suspicious behavior from normal administrative activity. Document why each false positive may occur. Recommend rule tuning options. Improve detection precision. Maintain MITRE ATT&CK mapping accuracy. Preserve visibility into high-risk behaviors. Support future detection improvement and analyst triage.

---

# 3. Scope of This Document

This document applies to custom Wazuh detections used in the Windows Detection Engineering Lab. It focuses on false-positive scenarios related to Windows endpoint behavior, Sysmon telemetry, Windows Event Logs, Windows Defender events, and Wazuh alert generation.

The detections reviewed in this document include:

| Detection ID | Detection Name | Wazuh Rule ID | MITRE ATT&CK Mapping |
|---|---|---|---|
| DET-001 | PowerShell Encoded Command Detection | 100100 | T1059.001 — PowerShell |
| DET-002 | Local User Creation Detection | 100101 | T1136.001 — Create Account: Local Account |
| DET-003 | Windows Service Creation / Modification | 100102 | T1543.003 — Windows Service |
| DET-009 | Defender / EICAR Detection | 100108 | T1204.002 — User Execution: Malicious File |
| DET-010 | Additional Windows Detection Rule | 100109 | Project-specific |

This document does not cover production tuning, enterprise allowlisting, threat intelligence integration, or live business environment exceptions. All examples are based on a controlled lab environment.

---

# 4. Lab Environment Summary

The lab environment consists of a Windows 10 endpoint connected to an Ubuntu 22.04 Wazuh server. The Windows endpoint runs Wazuh Agent and Sysmon. Controlled test activity is generated on the Windows endpoint and forwarded to the Wazuh manager for decoding, rule matching, indexing, and dashboard validation.

The detection pipeline follows this flow:

**Controlled Windows test activity → Sysmon / Windows Event Logs / Defender logs → Wazuh Agent → Wazuh Manager → Custom Wazuh Rule Match → Wazuh Dashboard Alert → Analyst Review → False Positive Tuning Notes**

The purpose of this project is to demonstrate detection engineering, validation, MITRE ATT&CK mapping, and analyst-style documentation in a safe local lab.

---

# 5. Definition of a False Positive

A false positive occurs when an alert is generated for activity that matches the detection logic but is not actually malicious or unauthorized.

In detection engineering, there are two important categories to understand:

**Technical false positive:**
The rule triggers on activity that should not have matched the intended detection logic.

**Benign true positive:**
The rule correctly detects the behavior, but the behavior is legitimate in context.

For example, if a rule detects local user creation and an administrator intentionally creates a local account during maintenance, the alert is technically correct, but the activity is benign. This should be documented as a benign true positive rather than a broken detection.

This distinction is important because not every non-malicious alert means the detection is bad. Some alerts are valid but require context.

---

# 6. Detection Tuning Principles

False-positive tuning should follow a careful and controlled process. A detection should not be disabled simply because it generated benign alerts during testing.

The following principles should guide tuning decisions:

First, confirm that the detection is matching the correct event source. Second, verify that the rule ID and MITRE mapping are accurate. Third, review the command line, parent process, user account, hostname, timestamp, and source event. Fourth, identify whether the alert is a true false positive or a benign true positive. Fifth, tune the rule only after understanding why the alert occurred. Sixth, avoid over-tuning the rule to the point where real suspicious activity would be missed. Seventh, document all exclusions, allowlists, severity changes, and logic changes.

The best detection tuning approach is to improve context, not blindly suppress alerts.

---

# 7. Tuning Decision Model

Each alert should be reviewed using the following decision model:

| Question | Purpose |
|---|---|
| Did the alert match the intended behavior? | Confirms whether the detection logic worked |
| Was the activity expected in the lab? | Determines whether it was part of validation |
| Was the command executed by an administrator or normal user? | Helps identify risk level |
| Was the parent process normal or suspicious? | Adds process-chain context |
| Was the behavior repeated across time or hosts? | Helps identify automation or malicious pattern |
| Did the alert map to the correct MITRE technique? | Confirms threat-informed mapping |
| Is the alert too broad? | Identifies tuning need |
| Would suppressing this alert hide real malicious behavior? | Prevents over-tuning |
| Is the detection still useful after tuning? | Ensures detection value remains |

---

# 8. False Positive Severity Categories

False positives should be categorized by operational impact.

| Category | Description | Recommended Action |
|---|---|---|
| Low Impact | Rare benign alert with clear context | Document only |
| Medium Impact | Recurring benign alert but still useful detection | Tune rule conditions or severity |
| High Impact | Frequent alert noise affecting analyst workflow | Add exclusions or correlation logic |
| Critical Impact | Alert causes service issues, incorrect rule matching, or dashboard noise | Roll back or disable specific rule temporarily |

---

## 9. High-Level False Positive Summary

| Detection ID | Detection Name | False Positive Risk | Tuning Priority |
|---|---|---|---|
| DET-001 | PowerShell Encoded Command Detection | Medium to High | High |
| DET-002 | Local User Creation Detection | Low to Medium | Medium |
| DET-003 | Windows Service Creation / Modification | Medium to High | High |
| DET-009 | Defender / EICAR Detection | Low | Low |
| DET-010 | Additional Windows Detection Rule | Unknown / Depends on logic | Pending |

---

# 10. Detection-Specific False Positive Tuning

---

## DET-001 — PowerShell Encoded Command Detection

## Detection Overview

DET-001 detects suspicious PowerShell encoded command execution. The rule is designed to identify PowerShell commands using encoded command arguments, which may be used by attackers to hide script content, bypass casual inspection, or execute obfuscated commands.

**Detection ID:** DET-001
**Rule ID:** 100100
**Technique:** T1059.001 — PowerShell
**Tactic:** Execution
**Primary Data Source:** Sysmon Event ID 1 — Process Creation
**Expected Test Activity:** Encoded PowerShell command executed on the Windows endpoint

---

## Why This Detection Matters

PowerShell is a powerful Windows scripting environment used by administrators and attackers. Because it is built into Windows, attackers often abuse PowerShell for execution, discovery, downloading payloads, running encoded commands, and interacting with system resources.

Encoded PowerShell is especially important because the `-EncodedCommand` or `-enc` parameter can hide the readable command from casual review. This makes it a useful detection point in SOC monitoring.

---

## Possible False Positives

The following benign activity may trigger this detection:

| False Positive Source | Explanation |
|---|---|
| IT automation scripts | Admin teams may use encoded PowerShell in deployment or automation workflows |
| Remote management tools | RMM tools may launch PowerShell with encoded commands |
| Software deployment platforms | SCCM, Intune, PDQ Deploy, or similar tools may execute PowerShell scripts |
| Security tools | EDR, antivirus, or monitoring tools may run PowerShell for checks |

| | |
|---|---|
| Lab testing | Controlled project testing intentionally triggers the rule |
| System administration | Administrators may use encoded scripts for compatibility or automation |
| Scheduled scripts | Recurring administrative jobs may use encoded PowerShell |

---

## Important Fields to Review

When investigating this alert, review the following fields:

| Field | Why It Matters |
|---|---|
| Image | Confirms whether powershell.exe or pwsh.exe executed |
| CommandLine | Shows encoded command flag and arguments |
| ParentImage | Helps identify what launched PowerShell |
| ParentCommandLine | Provides context for the process chain |
| User | Shows whether admin or normal user executed it |
| IntegrityLevel | Helps determine privilege context |
| CurrentDirectory | May reveal suspicious execution location |
| Hashes | Helps identify suspicious binaries |
| Hostname / Agent Name | Shows affected endpoint |
| Timestamp | Helps correlate with user activity or testing window |

---

## Common Benign Parent Processes

The following parent processes may indicate legitimate activity, but they should still be reviewed:

| Parent Process | Possible Benign Explanation |
|---|---|
| services.exe | Service-based management activity |
| taskeng.exe / taskhostw.exe | Scheduled task execution |

| | |
|---|---|
| explorer.exe | User-launched PowerShell |
| cmd.exe | Manual command-line execution |
| msiexec.exe | Installer or software deployment |
| powershell.exe | Nested PowerShell execution |
| IntuneManagementExtension.exe | Microsoft Intune management activity |
| SCCM client processes | Enterprise software deployment |

---

## Suspicious Indicators

The alert should be treated as more suspicious if one or more of the following are present:

PowerShell is launched by Microsoft Office applications, browser processes, archive tools, script interpreters, or unknown executables. The command includes `-EncodedCommand`, `-enc`, `-nop`, `-w hidden`, `-ExecutionPolicy Bypass`, `DownloadString`, `Invoke-WebRequest`, `IEX`, or suspicious remote URLs. The process runs from a temporary directory, downloads content, spawns additional processes, or connects to external IP addresses. The user is not an administrator but is executing administrative commands. Similar commands run repeatedly across multiple endpoints.

---

## Tuning Considerations

The rule should not alert on every PowerShell execution. It should focus on suspicious PowerShell patterns.

Recommended tuning options:

| Tuning Method | Purpose |
|---|---|
| Match encoded command flags | Focus rule on `-EncodedCommand`, `-enc`, or encoded payload patterns |
| Add suspicious flag matching | Include `-NoProfile`, `-ExecutionPolicy Bypass`, `-WindowStyle Hidden` |
| Add parent process context | Increase severity if parent is Office, browser, script host, or unknown process |

| | |
|---|---|
| Reduce severity for known admin tools | Lower alert level for approved automation platforms |
| Add user context | Prioritize non-admin users executing encoded PowerShell |
| Add frequency logic | Alert when repeated encoded PowerShell events occur in short time |
| Add destination context | Increase severity when PowerShell creates external network connections |
| Document lab test commands | Prevent confusing lab validation with unexpected activity |

---

## Recommended Rule Logic Improvement

Instead of alerting on all PowerShell activity, the rule should prioritize command lines containing suspicious execution indicators.

Recommended suspicious terms:

```
-EncodedCommand
-enc
-ExecutionPolicy Bypass
-NoProfile
-WindowStyle Hidden
Invoke-WebRequest
DownloadString
IEX
FromBase64String
```

---

## Tuning Recommendation

DET-001 should remain enabled because it is a high-value detection. However, the rule should be tuned to avoid alerting on normal PowerShell use. It should focus on encoded commands and suspicious execution flags.

**Recommended action:** Tune, do not remove.
**Recommended severity:** Medium to High depending on parent process and command context.
**False positive risk:** Medium to High.
**Priority for tuning:** High.

---

# DET-002 — Local User Creation Detection

## Detection Overview

DET-002 detects local Windows user account creation. This detection is mapped to account creation behavior that may be used for persistence or unauthorized access.

**Detection ID:** DET-002
**Rule ID:** 100101
**Technique:** T1136.001 — Create Account: Local Account
**Tactic:** Persistence
**Primary Data Source:** Windows Security Logs and Sysmon Process Creation
**Expected Test Activity:** `net user labuser P@ssw0rd123 /add`

---

## Why This Detection Matters

Unexpected local account creation is a valuable security signal. Attackers may create local users to maintain access, bypass normal authentication workflows, or create backup accounts for future use. Even though local account creation can be legitimate, it should be visible in a SOC environment.

---

## Possible False Positives

| False Positive Source | Explanation |
|---|---|
| IT onboarding | Admin creates a temporary or permanent local user |
| Helpdesk troubleshooting | Support creates local account for recovery or access |
| Lab setup | Test user accounts created during validation |
| Software installation | Some tools create local service or support accounts |
| Endpoint repair | Local admin account may be created for recovery |
| Imaging or provisioning | New endpoint build process may create local accounts |
| Security testing | Authorized testing intentionally creates users |

---

## Important Fields to Review

| Field | Why It Matters |
|---|---|
| Target username | Identifies the newly created account |
| Creator username | Shows who created the account |
| Hostname | Identifies affected endpoint |
| CommandLine | Shows account creation command |
| ParentProcess | Helps identify whether it was manual or automated |
| Group membership | Determines whether account was added to administrators |
| Timestamp | Helps compare with approved change windows |
| Logon session | Helps identify user context |

---

## Suspicious Indicators

This detection becomes more suspicious when the new account has an unusual name, is created outside business hours, is immediately added to the local Administrators group, is created by a non-admin or unexpected user, appears on multiple endpoints, or is followed by remote login attempts, service creation, scheduled task creation, or persistence activity.

Examples of suspicious account names may include generic or deceptive names such as `backupadmin`, `supportuser`, `admin2`, `svc-update`, `tempadmin`, or names that look similar to legitimate accounts.

---

## Tuning Considerations

DET-002 should not be tuned too aggressively because local account creation is generally important. However, context should be added to improve prioritization.

Recommended tuning options:

| Tuning Method | Purpose |
|---|---|
| Monitor account creation plus admin group addition | Higher fidelity than account creation alone |

| | |
|---|---|
| Add maintenance window context | Reduce priority during approved testing or provisioning |
| Add known admin user context | Lower severity if performed by expected administrator |
| Add endpoint role context | Differentiate lab endpoint, server, and workstation behavior |
| Add frequency logic | Alert if multiple accounts are created quickly |
| Correlate with logon events | Identify whether the account was used after creation |
| Correlate with service creation | Higher severity if account creation is followed by service persistence |

---

## Recommended Rule Logic Improvement

A stronger detection would alert at a medium level for local user creation and increase severity if the account is added to a privileged group.

Example logic idea:

Local user created = Medium severity
Local user created + added to Administrators group = High severity
Local user created + remote login activity = High severity
Local user created + service creation = High severity

---

## Cleanup Consideration

Any test account created during validation should be removed after testing.

Example cleanup command:

```
net user labuser /delete
```

The cleanup should be documented in the validation notes.

---

## Tuning Recommendation

DET-002 should remain enabled. It is a valuable detection with moderate false-positive risk. The best tuning approach is to add context rather than suppress account creation alerts.

**Recommended action:** Keep enabled with contextual tuning.
**Recommended severity:** Medium, upgraded to High if privileged group membership is added.
**False positive risk:** Low to Medium.
**Priority for tuning:** Medium.

---

# DET-003 — Windows Service Creation / Modification Detection

## Detection Overview

DET-003 detects Windows service creation or modification. This behavior is mapped to attackers creating or modifying Windows services for persistence, execution, or privilege escalation.

**Detection ID:** DET-003
**Rule ID:** 100102
**Technique:** T1543.003 — Create or Modify System Process: Windows Service
**Tactic:** Persistence / Privilege Escalation
**Primary Data Source:** Sysmon Event ID 1 and Windows System Logs
**Expected Test Activity:** `sc.exe create LabService binPath= "C:\Windows\System32\cmd.exe /c whoami"`

---

## Why This Detection Matters

Windows services can run with elevated privileges and start automatically. Attackers may create malicious services to maintain persistence or execute payloads. Service creation is also a common activity during software installation, which means false-positive tuning is very important.

---

## Possible False Positives

| False Positive Source | Explanation |
|---|---|
| Software installation | New applications commonly create services |

| | |
|---|---|
| Driver installation | Hardware or software drivers may create services |
| Windows updates | Updates may modify or create system services |
| Security tools | Antivirus, EDR, VPN, and backup agents often create services |
| IT management tools | Remote management platforms may install services |
| Printer or network software | Some utilities create background services |
| Lab testing | Controlled `sc.exe create` command triggers the detection |

---

## Important Fields to Review

| Field | Why It Matters |
|---|---|
| Service name | Identifies the created or modified service |
| Service binary path | Shows what the service executes |
| CommandLine | Shows full service creation command |
| Parent process | Helps identify installer vs suspicious process |
| User | Shows who created the service |
| Image | Confirms whether sc.exe or another tool created it |
| Timestamp | Helps compare with known installation activity |
| File path | Suspicious if service points to Temp, Downloads, AppData, or unusual paths |
| Digital signature | Helps determine legitimacy of service binary |

---

## Suspicious Indicators

Service creation should be treated as more suspicious when the service binary path points to unusual directories such as `Temp`, `Downloads`, `AppData`, `Public`, user profile paths, or unknown executable locations. It is also suspicious if the service executes `cmd.exe`, `powershell.exe`, `wscript.exe`, `cscript.exe`, `mshta.exe`, or an unsigned binary.

Other suspicious indicators include service creation by a non-admin user, service names that mimic Windows services, service creation followed by network connections, service creation outside maintenance windows, or multiple service creation events in a short time.

---

## Tuning Considerations

DET-003 has a higher false-positive risk than local user creation because many legitimate software installations create services.

Recommended tuning options:

| Tuning Method | Purpose |
|---|---|
| Allowlist known service names | Reduce alerts from approved software |
| Allowlist signed vendor binaries | Reduce alerts from trusted software |
| Increase severity for suspicious paths | Prioritize services running from Temp, AppData, Downloads |
| Increase severity for script interpreters | Prioritize services executing cmd, PowerShell, mshta, wscript |
| Add parent process context | Differentiate installer activity from suspicious command-line activity |
| Add maintenance window awareness | Reduce expected software installation noise |
| Correlate with account creation | Higher severity if new account and new service appear together |
| Correlate with network activity | Higher severity if new service creates external connections |

---

## Recommended Rule Logic Improvement

A basic service creation rule may be too broad. A better version should use layered severity.

Suggested severity model:

Service creation by trusted installer = Low or informational
Service creation by administrator using known software = Medium

Service creation using `sc.exe` manually = Medium
Service creation pointing to script interpreter = High
Service creation from Temp/AppData/Public folder = High
Service creation followed by network activity = High

---

### Cleanup Consideration

Any test service created during validation should be removed after testing.

Example cleanup command:

```
sc.exe delete LabService
```

The cleanup status should be recorded in the validation matrix.

---

### Tuning Recommendation

DET-003 should remain enabled because it is a high-value persistence detection. However, it requires careful tuning because legitimate service creation is common.

**Recommended action:** Keep enabled but tune carefully.
**Recommended severity:** Medium by default, High for suspicious paths or interpreter-based service commands.
**False positive risk:** Medium to High.
**Priority for tuning:** High.

---

## DET-009 — Defender / EICAR Detection

### Detection Overview

DET-009 detects Windows Defender activity related to the EICAR antivirus test file. EICAR is a safe test file used to validate antivirus detection and alert forwarding.

**Detection ID:** DET-009
**Rule ID:** 100108
**Technique:** T1204.002 — User Execution: Malicious File
**Tactic:** Execution
**Primary Data Source:** Windows Defender Logs / Windows Security Logs
**Expected Test Activity:** Defender detects EICAR test file

---

## Why This Detection Matters

This detection validates that endpoint protection telemetry can flow from Windows Defender into Wazuh. The purpose is not to test real malware, but to confirm that security tool alerts are visible in the SIEM.

This is important for SOC workflows because endpoint protection alerts often provide early evidence of suspicious files, blocked downloads, or user-executed threats.

---

## Possible False Positives

| False Positive Source | Explanation |
|---|---|
| Security team testing | EICAR is intentionally used for validation |
| Lab testing | Project validation intentionally triggers Defender |
| Antivirus health checks | Some tools may test detection pipelines |
| Repeated file restore | Same file may be repeatedly detected |
| User testing | A user may download EICAR from a public test page |

---

## Important Fields to Review

| Field | Why It Matters |
|---|---|
| Threat name | Confirms EICAR or Defender detection name |
| File path | Shows where the test file was created |
| Action taken | Shows blocked, quarantined, removed, or allowed |
| User | Identifies who created or downloaded the file |
| Hostname | Identifies affected endpoint |
| Detection time | Helps correlate with test activity |
| Source process | Identifies browser, PowerShell, or manual file creation |

---

## Suspicious Indicators

For EICAR specifically, this should usually be treated as validation activity. However, Defender detections should be treated as more serious when the detected file is not EICAR, when the file path is unusual, when multiple detections appear, when Defender fails to remove the file, when detection is followed by execution activity, or when the source process is suspicious.

---

## Tuning Considerations

DET-009 has a low false-positive risk in this project because EICAR is intentionally used for validation. However, the rule should clearly distinguish between EICAR test detections and real Defender malware alerts.

Recommended tuning options:

| Tuning Method | Purpose |
|---|---|
| Match EICAR-specific detection name | Prevent confusing EICAR tests with real malware |
| Label EICAR alerts as validation | Make lab testing context clear |
| Separate EICAR from real malware detections | Preserve severity for real Defender events |
| Include Defender action taken | Identify whether Defender blocked or quarantined file |
| Document test window | Avoid confusion during future review |

---

## Recommended Rule Logic Improvement

EICAR-related detections should be labeled as safe validation events. Real Defender malware detections should remain separate and should have higher severity.

Suggested classification:

EICAR detection = Lab validation / Medium
Real Defender malware detection = High
Defender blocked threat = High
Defender allowed threat or remediation failed = Critical

---

### Cleanup Consideration

The EICAR test file should be removed or allowed to be quarantined by Defender. The validation notes should state that no real malware was used.

---

### Tuning Recommendation

DET-009 should remain enabled as a pipeline validation rule. It should be clearly documented as a safe antivirus telemetry test.

**Recommended action:** Keep enabled for validation.
**Recommended severity:** Medium for EICAR, High for real Defender malware detections.
**False positive risk:** Low.
**Priority for tuning:** Low.

---

## DET-010 — Additional Windows Detection Rule

### Detection Overview

DET-010 is reserved for an additional Windows detection in the project. Since the final logic may vary, false-positive tuning depends on what behavior the rule detects.

**Detection ID:** DET-010
**Rule ID:** 100109
**Technique:** Project-specific
**Tactic:** Project-specific
**Primary Data Source:** Sysmon / Windows Event Logs
**Validation Status:** Pending or optional

---

### Possible False Positives

Because DET-010 is project-specific, possible false positives should be documented after the test behavior is finalized. Common sources may include administrator activity, endpoint management tools, software installation, normal Windows background processes, lab testing commands, or expected system behavior.

---

### Tuning Considerations

Before marking DET-010 as validated, the following should be reviewed:

What exact behavior does the rule detect? Which Windows event or Sysmon event is used? What command or action triggers it? Is the MITRE mapping accurate? Does the behavior occur normally in Windows? Could administrators trigger it during normal work? Does the rule duplicate an existing Wazuh rule? Does the rule generate too many alerts? Can it be improved with parent process, user, command line, path, hash, or frequency context?

---

### Recommended Tuning Approach

DET-010 should not be considered complete until it has documented false-positive notes, validation evidence, and cleanup steps.

**Recommended action:** Keep pending until fully validated.
**Recommended severity:** Depends on behavior.
**False positive risk:** Unknown.
**Priority for tuning:** Pending.

---

# 11. Cross-Detection False Positive Patterns

Some false positives may affect multiple detections.

| Pattern | Affected Detections | Explanation |
|---|---|---|
| Administrator testing | DET-001, DET-002, DET-003 | Admins often run PowerShell, create users, or modify services |
| Lab validation | All detections | Controlled testing intentionally triggers alerts |
| Software installation | DET-003 | Services may be created during installation |
| Security tooling | DET-001, DET-003, DET-009 | EDR, Defender, or management tools may trigger alerts |

| | | |
|---|---|---|
| Scheduled maintenance | DET-002, DET-003 | Admin changes during maintenance windows may be expected |
| Automation scripts | DET-001 | Encoded or scripted PowerShell may be legitimate |
| Endpoint provisioning | DET-002, DET-003 | New endpoint setup may create users/services |

---

# 12. Recommended Tuning Fields

When tuning detections, the following fields are useful:

| Field | Use |
|---|---|
| agent.name | Identify affected endpoint |
| rule.id | Confirm custom detection |
| rule.description | Confirm alert purpose |
| data.win.eventdata.image | Identify executable |
| data.win.eventdata.commandLine | Review command-line arguments |
| data.win.eventdata.parentImage | Identify parent process |
| data.win.eventdata.parentCommandLine | Understand process chain |
| data.win.eventdata.user | Identify user context |
| data.win.eventdata.targetObject | Review registry path |
| data.win.eventdata.serviceName | Review service name |
| data.win.eventdata.serviceFileName | Review service binary path |
| data.win.system.eventID | Confirm Windows event ID |
| sysmon.event_id | Confirm Sysmon event type |
| timestamp | Correlate with testing or maintenance window |

---

# 13. Recommended Tuning Methods

False positives can be reduced using several methods.

## 13.1 Add Specific Matching Conditions

A broad rule should be narrowed with stronger conditions. For example, instead of detecting all PowerShell execution, detect PowerShell with suspicious flags.

## 13.2 Add Parent Process Context

Parent process context helps distinguish normal administrative activity from suspicious execution chains.

Example suspicious parent processes include Office apps, browsers, script hosts, archive tools, and unknown executables.

## 13.3 Add User Context

Alerts from normal administrative users may be lower severity than alerts from standard users. However, admin activity should not be ignored completely because compromised admin accounts are high risk.

## 13.4 Add Path Context

Suspicious file paths can increase severity. Examples include Temp, AppData, Downloads, Public, and unusual user profile directories.

## 13.5 Add Frequency or Timeframe Logic

A single discovery command may be normal. Multiple discovery commands within a short period may be suspicious.

## 13.6 Add Allowlists Carefully

Allowlists should be used carefully and only for known legitimate processes, users, service names, or signed software. Broad allowlists can hide real attacks.

## 13.7 Adjust Alert Severity

Not every alert needs to be high severity. Severity should reflect confidence and potential impact.

### 13.8 Document Known Benign Activity

Expected lab activity should be documented so that future review does not confuse validation alerts with unexpected behavior.

---

## 14. Detection Severity Tuning Recommendations

| Detection ID | Current/Expected Severity | Recommended Severity Logic |
|---|---|---|
| DET-001 | High or Medium | Medium for encoded PowerShell, High if suspicious parent process or external connection exists |
| DET-002 | Medium | Medium for user creation, High if user is added to Administrators group |
| DET-003 | Medium or High | Medium for service creation, High if service path is suspicious or uses script interpreter |
| DET-009 | Medium | Medium for EICAR validation, High for real Defender malware detections |
| DET-010 | Pending | Severity should be based on detection behavior and confidence |

---

## 15. False Positive Investigation Checklist

For every alert, the analyst should ask:

| Check | Answer |
|---|---|
| Was this alert generated during controlled lab testing? | Yes / No |

| | |
|---|---|
| Does the alert match the correct rule ID? | Yes / No |
| Does the alert map to the correct MITRE technique? | Yes / No |
| Was the command expected? | Yes / No |
| Was the user expected? | Yes / No |
| Was the parent process expected? | Yes / No |
| Was the host expected? | Yes / No |
| Is this behavior normal for this endpoint? | Yes / No |
| Is the alert too broad? | Yes / No |
| Should the rule be tuned, severity-adjusted, or documented only? | Tune / Adjust / Document |
| Is cleanup required? | Yes / No |
| Was evidence captured? | Yes / No |

---

# 16. Rule Tuning Documentation Template

Each tuning change should be documented using the following template.

**Tuning ID:** TUNE-[Number]
**Date:** [Insert Date]
**Detection ID:** [DET-001 / DET-002 / DET-003 / DET-009 / DET-010]
**Rule ID:** [Insert Rule ID]
**Original Issue:** [Describe false positive or alert noise]
**Observed Trigger:** [Command, event, process, or user]
**False Positive Type:** [Technical false positive / Benign true positive]
**Tuning Action:** [Add condition / Reduce severity / Add exclusion / Add context / Disable temporarily]
**Fields Used for Tuning:** [CommandLine, ParentImage, User, ServiceName, etc.]
**Expected Result After Tuning:** [Describe expected behavior]
**Validation Result:** [Passed / Failed / Pending]
**Evidence:** [Screenshot or JSON reference]
**Analyst Notes:** [Additional details]

---

# 17. Example Tuning Entry

**Tuning ID:** TUNE-001
**Date:** [Insert Date]
**Detection ID:** DET-001
**Rule ID:** 100100
**Original Issue:** PowerShell encoded command detection generated alerts during controlled lab testing and may also trigger on legitimate administrative automation.
**Observed Trigger:** `powershell.exe -EncodedCommand dwBoAG8AYQBtAGkA`
**False Positive Type:** Benign true positive in lab context.
**Tuning Action:** Keep rule enabled, but document lab test activity and recommend additional parent process review before escalation.
**Fields Used for Tuning:** CommandLine, ParentImage, User, Agent Name, Timestamp.
**Expected Result After Tuning:** Alert remains visible but is interpreted correctly based on context.
**Validation Result:** Passed.
**Evidence:** powershell-encoded-command-alert.json and dashboard screenshot.
**Analyst Notes:** Detection is useful but should be correlated with suspicious parent process or network activity before being treated as high confidence malicious activity.

---

# 18. Suppression and Allowlisting Guidelines

Suppression should be used only when the activity is confirmed as safe, recurring, and well understood.

Acceptable allowlist examples include known management tools, approved software deployment processes, documented lab test accounts, known service names from trusted vendors, and approved security testing windows.

Unsafe allowlist examples include all PowerShell activity, all administrator activity, all service creation, all Defender detections, all commands from a specific user, or all activity from the Windows endpoint.

Overly broad allowlists can make the detection useless. Every allowlist should be specific, documented, and reviewed later.

---

# 19. Recommended Review Cycle

False-positive tuning should not be a one-time activity. Detections should be reviewed after initial validation and again after additional test data is collected.

Recommended review schedule for this project:

| Review Stage | Purpose |
|---|---|
| Initial validation | Confirm rule triggers correctly |
| First tuning review | Identify obvious false positives |
| Post-cleanup review | Confirm test artifacts are removed |
| Final documentation review | Ensure evidence and notes are complete |
| Portfolio review | Ensure wording is accurate and professional |

---

# 20. False Positive Risk Matrix

| Detection ID | Likelihood of False Positives | Impact of False Positives | Overall FP Risk | Tuning Need |
|---|---|---|---|---|
| DET-001 | High | Medium | High | Strong tuning needed |
| DET-002 | Medium | Medium | Medium | Contextual tuning needed |
| DET-003 | High | High | High | Strong tuning needed |
| DET-009 | Low | Low | Low | Minimal tuning needed |
| DET-010 | Unknown | Unknown | Pending | Requires validation |

---

# 21. Detection Confidence Matrix

| Detection ID | Detection Confidence | Reason |
|---|---|---|
| DET-001 | Medium | Encoded PowerShell is suspicious but may be used by automation |
| DET-002 | High | Local user creation is a clear and meaningful event |
| DET-003 | Medium | Service creation is important but common during legitimate installs |
| DET-009 | High for EICAR validation | Defender detection confirms telemetry flow, but EICAR is lab-only |
| DET-010 | Pending | Confidence depends on final detection logic |

---

# 22. Recommended Improvements for Future Version

Future improvements to this project could include the following:

Add parent-child process chain analysis for PowerShell detections. Add severity escalation when suspicious PowerShell creates network connections. Correlate local user creation with group membership changes. Correlate service creation with suspicious binary paths. Add known-good service allowlist for Windows and trusted software. Add detection for scheduled task creation. Add detection for registry Run key persistence. Add separate severity for EICAR test versus real Defender malware detections. Add alert grouping by host and timeframe. Add a tuning log for every rule change. Add test cases for benign activity and suspicious activity separately.

These improvements would make the project more mature and closer to real-world SOC detection engineering practice.

---

# 23. Analyst Notes by Detection

## DET-001 Analyst Note

This detection should be reviewed carefully because PowerShell is used heavily in both administration and attacks. Encoded PowerShell should not be ignored, but it should be evaluated using context. The strongest suspicious indicators are encoded commands combined with suspicious parent processes, execution policy bypass, hidden windows, external downloads, or unusual user activity.

## DET-002 Analyst Note

Local user creation is a strong detection because account creation is easy to understand and investigate. The most important tuning improvement is to distinguish normal admin activity from unexpected account creation. If the account is added to privileged groups, the alert should be treated as higher severity.

## DET-003 Analyst Note

Windows service creation is high-value but noisy. Many legitimate applications create services. The most useful tuning is based on service binary path, signer, parent process, command line, and whether the service launches a script interpreter or unknown executable.

## DET-009 Analyst Note

EICAR should be clearly documented as a safe validation test. The detection proves that Defender telemetry reaches Wazuh, but it should not be presented as real malware execution. Real Defender malware detections should be handled separately with higher severity.

## DET-010 Analyst Note

DET-010 should not be finalized until the detection logic, MITRE mapping, test command, evidence, false positives, and cleanup procedure are fully documented.

---

# 24. Final False Positive Tuning Summary

The custom detections created for the Windows Detection Engineering Lab provide meaningful endpoint visibility and demonstrate practical detection engineering skills. However, each detection has different false-positive risks.

DET-001 and DET-003 require the most tuning because PowerShell and Windows service creation are both commonly used by administrators and attackers. DET-002 has strong detection value and moderate false-positive risk because local user creation is less frequent but may occur during legitimate administration. DET-009 has low false-positive risk in this lab because it is tied to EICAR validation, but it should be clearly labeled as safe test activity. DET-010 should remain pending until its final logic is validated.

The recommended tuning approach is to preserve detection coverage while adding context through command-line analysis, parent process review, user context, file path review, severity adjustment, and documented false-positive handling.

---

# 25. Professional Portfolio Statement

This false-positive tuning document demonstrates that the project goes beyond basic alert creation. It shows an understanding of detection quality, analyst workload, rule tuning, SOC investigation context, and the importance of reducing alert noise without losing visibility.

In a real SOC environment, detection engineering is not complete when an alert fires. A detection must also be understandable, testable, tunable, and useful for analysts. This document supports that professional standard by documenting likely false positives, tuning logic, severity considerations, and investigation guidance for each custom detection.

---

# 26. Conclusion

False-positive tuning is an essential part of the detection engineering lifecycle. The detections in this project are valuable because they identify behaviors commonly associated with Windows-based attack techniques, including encoded PowerShell execution, local account creation, service creation, and Defender malware-test telemetry.